

VA ÄRENDEHANTERING

Teknisk Dokumentation

Systemarkitektur, datamodell, API-kontrakt och utökningsmönster

Stack: Next.js 15 · Supabase · TypeScript · Tailwind CSS

Version 1.0 · 2026 · Intern – Konfidentiell

Innehåll

- 1. Systemöversikt** — Syfte, aktörer, högnivåarkitektur
- 2. Teknisk stack** — Alla bibliotek, versioner och motivering
- 3. Katalogstruktur** — Fullständigt filträd med förklaringar
- 4. Datamodell** — ER-diagram i text, alla tabeller och kolumner
- 5. Autentisering och auktorisering** — Auth-flöde, RLS, roller, middleware
- 6. API-lager** — Server Actions, Route Handlers, kontrakt
- 7. Integrationer** — Gmail (MCP), OneDrive (Graph API), Resend
- 8. Edge Functions** — Alla Supabase Functions, triggers, payloads
- 9. Frontend-arkitektur** — Komponenthierarki, state, realtid
- 10. Säkerhetsmodell** — RLS-policies, hemlighetshantering, JWT
- 11. Utökningsmönster** — Hur man lägger till features, tabeller, routes
- 12. Miljövariabler** — Komplet referens med kontext
- 13. Kända begränsningar** — Tech debt, saknade features, framtida arbete

1. Systemöversikt

1.1 Syfte

VA Ärendehantering är ett SaaS-liknande projekthanteringssystem byggt specifikt för en liten VA-konsultverksamhet (vatten/avlopp). Systemet hanterar 15–30 simultaneously aktiva projekt med tre primära domäner:

- Intern projekthantering – projekt, ärenden, milstolpar, kontakt-CRM
- Extern kommunikation – kundportal per projekt med realtidschatt och dokumentdelning
- Integrationer – Gmail (ärendekoppling), OneDrive/Microsoft 365 (dokumentlagring), Resend (e-postnotifikationer)

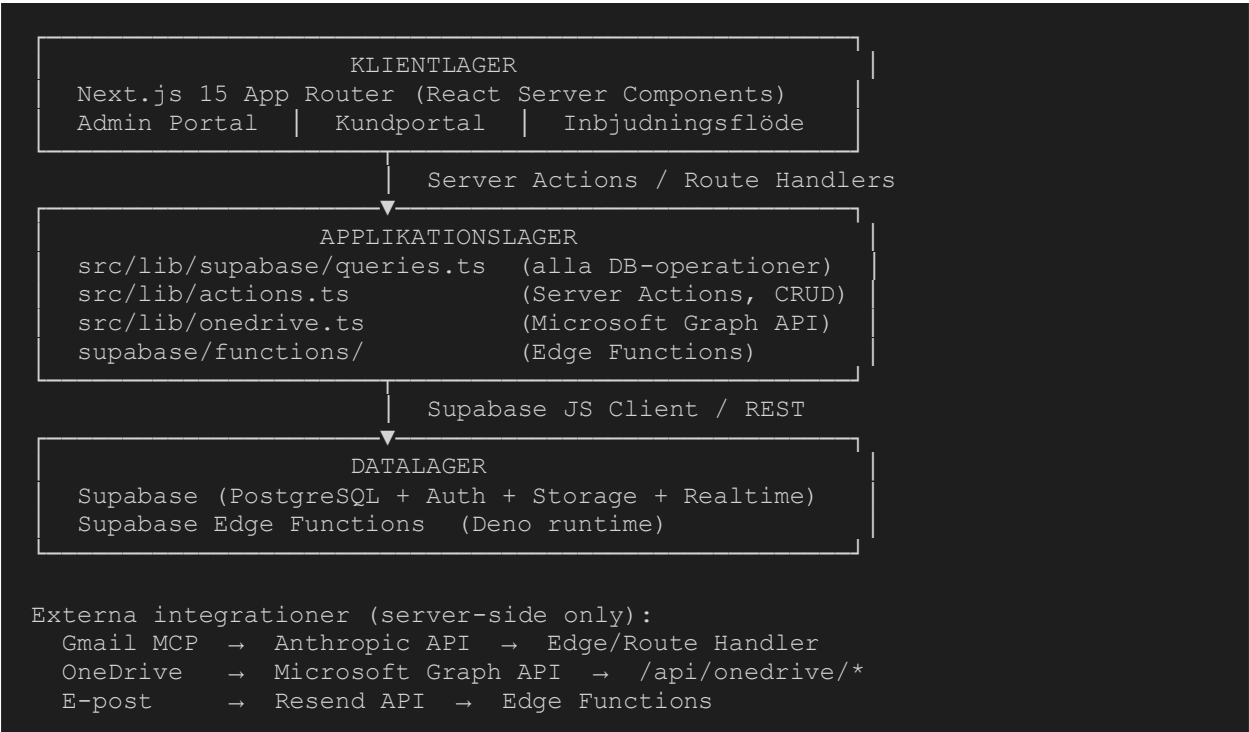
1.2 Aktörer

Aktör	Beskrivning
Admin	Konsulten själv. Full tillgång till alla moduler. En enda admin-användare i nuläget.

Kund	Fastighetsägare, entreprenör, samfällighet eller företag. Begränsad vy – ser endast sitt/sina projekt via kundportalen.
System	Supabase Edge Functions och pg_cron som agerar autonomt (t.ex. dagliga e-postpåminnelser).

1.3 Högnivåarkitektur

Systemet följer en klassisk tre-lagersarkitektur men med serverless-orienterad implementering:



1.4 Deployment-topologi

Komponent	Plattform / Tjänst
Next.js-app	Vercel (Serverless Functions + Edge Network)
Databas	Supabase (managed PostgreSQL, eu-central-1)
Auth	Supabase Auth (JWT, bcrypt, magic links)
Fyllagring	Supabase Storage (S3-kompatibel bucket)
Realtid	Supabase Realtime (WebSocket, postgres_changes)
Edge Functions	Supabase Edge Runtime (Deno)
E-post	Resend (SMTP-abstraktionslager)
Kodlagring	GitHub (CI/CD-trigger till Vercel)

2. Teknisk stack

2.1 Kärnberoenden

Paket / Tjänst	Version & motivering
next	15.2.0 – App Router, RSC, Server Actions, TypedRoutes
react / react-dom	19.0.0 – Concurrent features, useTransition, useOptimistic
@supabase/supabase-js	2.49.1 – Klient för DB, Auth, Storage, Realtime
@supabase/ssr	0.5.2 – Cookie-baserad auth för Next.js App Router
typescript	5.x – Strikt typing genom hela stacken
tailwindcss	3.4.1 – Utility-first CSS, custom design tokens

2.2 Dev-beroenden och verktyg

Verktyg	Syfte
eslint + eslint-config-next	Linting med Next.js-specifika regler
autoprefixer + postcss	CSS-preprocessning för Tailwind
supabase CLI	Lokal emulering, migrations, function deploy

2.3 Externa API:er

API	Protokoll / Auth
Anthropic API (Claude)	HTTPS POST / API Key – Gmail MCP-proxy
Microsoft Graph API	OAuth 2.0 Authorization Code + Refresh Token
Resend API	HTTPS POST / Bearer Token
Supabase REST API	HTTPS / JWT (anon eller service_role)
Supabase Realtime	WebSocket / JWT

3. Katalogstruktur

Komplett filträd med kortfattad förklaring av varje fils ansvar.

```

va-system/
├─ next.config.js      # Next.js-konfig: typedRoutes, bilddomäner
├─ tailwind.config.js  # Tailwind: fontFamily-tokens, brand-färger
├─ tsconfig.json       # TS: strikt läge, path alias @/*
├─ postcss.config.js   # PostCSS-pipeline
├─ package.json
├─ .env.local.example  # Mall för miljövariabler (checkas in)
├─ src/
│   └─ middleware.ts    # Auth-guard: skyddar /dashboard, /projects
etc.
├─
│   └─
│       └─ app/
│           └─ layout.tsx # Next.js App Router
│               └─ # Root layout: fonter (DM Sans + Playfair),
├─ metadata
│   └─ globals.css       # Tailwind base + globala CSS-variabler + .btn-
├─ primary etc.
│   └─ page.tsx          # Root → redirect till /dashboard
│       └─ login/
│           └─ page.tsx   # Admin-inloggning (Client Component, Supabase
signIn)
│   └─ (admin)/          # Route group - delar AdminLayout
│       └─ layout.tsx     # Verifierar admin-roll, renderar Sidebar
│           └─ dashboard/page.tsx # RSC: statistik, öppna ärenden,
försenade
│   └─ projects/
│       └─ page.tsx       # RSC: projektlista med filter
│           └─ new/page.tsx # RSC: formulär → createProjectAction
│               └─ [id]/
│                   └─ page.tsx # RSC: detaljvy, milstolpar, ärenden,
dokument
│   └─ edit/
│       └─ page.tsx       # RSC: redigeringsformulär
│           └─ MilestoneToggle.tsx # CC: optimistisk toggle via
useTransition
│   └─ contacts/
│       └─ page.tsx       # RSC: kontaktkort med roller
│           └─ new/page.tsx # RSC: kontaktformulär
│               └─ [id]/
│                   └─ page.tsx # RSC: redigera, projektkopplingar,
portalinbjudan
│   └─ InviteButton.tsx   # CC: generera token + anropa send-
invite
│   └─ cases/
│       └─ page.tsx       # RSC: ärendelista, filter
status/prioritet
│   └─ new/page.tsx       # RSC: ärendeformulär med
projektkoppling
│   └─ [id]/
│       └─ page.tsx       # RSC: ärendedetalj +
redigeringsformulär
│   └─ StatusChanger.tsx  # CC: snabbändra status utan
formulär
│   └─ email/page.tsx     # CC: Gmail via MCP, trådar, koppling,
svar

```

```

├── documents/
│   ├── page.tsx          # RSC: OneDrive-filvy per projekt
│   └── OneDriveUploader.tsx # CC: drag-drop upload till
/api/onedrive/upload
├── portal/                # Kundportal
│   ├── layout.tsx        # Verifierar customer-roll
│   └── page.tsx          # RSC: projektväljare (auto-redirect vid
ett projekt)
├── login/page.tsx        # CC: kundportal-inloggning (teal-tema)
├── invite/[token]/
│   └── page.tsx          # CC: acceptera inbjudan, skapa konto
├── project/[id]/
│   └── layout.tsx        # Verifierar portal_access, renderar
PortalNav
├── PortalNav.tsx         # CC: flikar
(Översikt/Ärenden/Dokument/Meddelanden)
├── page.tsx             # RSC: milstolpar, framsteg, kontakt
├── cases/page.tsx       # RSC: expanderbara ärenden, DIN ÅTGÄRD-
märkning
├── documents/
│   ├── page.tsx          # RSC: filer med signerade URL:er
│   └── DocumentUploader.tsx # CC: drag-drop →
/api/documents/upload
├── messages/
│   └── page.tsx          # RSC: laddar initialMessages +
senderName
├── MessagesClient.tsx    # CC: Realtime-chatt, Supabase
channel
├── api/                  # Route Handlers (server-side)
│   ├── auth/
│   │   ├── signout/route.ts # POST → signOut + redirect
│   │   └── onedrive/callback/route.ts # GET: OAuth callback, sparar
refresh_token
├── gmail/send/route.ts   # POST: skickar Gmail-svar via
MCP
├── documents/upload/route.ts # POST: multipart → Supabase
Storage
├── onedrive/upload/route.ts # POST: multipart → Microsoft
Graph PUT
├── lib/
│   ├── supabase/
│   │   ├── types.ts      # TypeScript-interfaces för alla DB-entiteter
│   │   └── client.ts      # createBrowserClient() - används i Client
Components
├── server.ts            # createServerClient() - cookies, Server
Components/Actions
├── queries.ts           # Alla DB-operationer (select, insert, update,
delete)
├── actions.ts           # Next.js Server Actions: CRUD + revalidatePath
+ redirect
├── onedrive.ts          # Microsoft Graph: auth, listFiles, uploadFile,
getDownloadUrl
├── components/
│   ├── admin/
│   │   └── Sidebar.tsx    # CC: navigation, aktiv route, logout
│   └── ui/
│       └── FormFields.tsx # CC: SubmitButton (useFormStatus), Field,
TextInput, Select...
└── supabase/
    └── functions/

```

```
├─ shared/
│   └─ email.ts          # Deno: sendEmail() + fyra HTML-mailmallar
├─ notify-overdue/index.ts # Schemalagd: försenade ärenden →
kontakter
├─ notify-case-update/index.ts # Webhook: ärendestatus ändrad →
kontakt
├─ notify-new-message/index.ts # Webhook: nytt meddelande → admin
eller kund
└─ send-invite/index.ts      # HTTP: generera token + skicka
välkomstmejl
```

4. Datamodell

4.1 ER-diagram (textformat)

```
auth.users (Supabase-hanterad)
├── < profiles (1:1) role: admin | customer
├── < portal_access (M:M via contact_id + project_id)
│   ├── contacts
│   └── projects
└── projects
    ├── < milestones (1:N)
    ├── < cases (1:N)
    │   ├── contacts (N:1, nullable)
    │   └── < case_comments (1:N)
    ├── < documents (1:N)
    ├── < email_links (1:N)
    ├── < messages (1:N)
    └── < project_contacts (M:M junction)
        └── contacts
```

4.2 Tabellschema

profiles

Kolumn	Typ / Constraint
id	uuid PK → auth.users(id) CASCADE
full_name	text nullable
role	text default 'admin' — 'admin' 'customer'
created_at	timestampz default now()

projects

Kolumn	Typ / Constraint
id	uuid PK default uuid_generate_v4()
name	text NOT NULL
type	project_type ENUM — VA-utredning Besiktning Rådgivning Tillstånd
status	project_status ENUM — Förfrågan Aktiv Granskning Avslutat
client	text NOT NULL
description	text nullable
deadline	date nullable
created_at / updated_at	timestampz (updated_at via trigger set_updated_at)

milestones

Kolumn	Typ / Constraint
id	uuid PK
project_id	uuid FK → projects(id) CASCADE
label	text NOT NULL
due_date	date nullable
done	boolean default false
sort_order	integer default 0

contacts

Kolumn	Typ / Constraint
id	uuid PK
name	text NOT NULL
role	contact_role ENUM — Fastighetsägare Entreprenör Samfällighet Företag
email	text nullable
phone	text nullable
company	text nullable
notes	text nullable

project_contacts (junction)

Kolumn	Typ / Constraint
project_id	uuid FK → projects(id) CASCADE
contact_id	uuid FK → contacts(id) CASCADE
(PK)	(project_id, contact_id) composite

portal_access

Kolumn	Typ / Constraint
id	uuid PK
contact_id	uuid FK → contacts(id) CASCADE
user_id	uuid FK → auth.users(id) SET NULL nullable
project_id	uuid FK → projects(id) CASCADE
invite_token	text UNIQUE default encode(gen_random_bytes(24), 'hex')
accepted_at	timestampz nullable — null = inbjudan ej accepterad

cases

Kolumn	Typ / Constraint
id	uuid PK

project_id	uuid FK → projects(id) CASCADE
contact_id	uuid FK → contacts(id) SET NULL nullable
title	text NOT NULL
description	text nullable
priority	case_priority ENUM — Hög Medium Låg
status	case_status ENUM — Öppen Pågående Stängd
assigned_to	text nullable — fri text, t.ex. 'Konsult'
deadline	date nullable
created_at / updated_at	timestampz (trigger)

documents

Kolumn	Typ / Constraint
id	uuid PK
project_id	uuid FK → projects(id) CASCADE
name	text NOT NULL — originalfilnamn
storage_path	text NOT NULL — {project_id}/{timestamp}_{name} i Supabase Storage
mime_type	text nullable
size_bytes	bigint nullable
category	text nullable — 'Rapport' 'Protokoll' 'Uppladdad' etc.
uploaded_by	uuid FK → auth.users SET NULL nullable
uploaded_by_name	text nullable
source	text default 'portal' — 'portal' 'onedrive'
onedrive_item_id	text nullable — Microsoft Graph item ID

email_links

Kolumn	Typ / Constraint
id	uuid PK
gmail_thread_id	text NOT NULL
gmail_message_id	text nullable
project_id	uuid FK → projects(id) CASCADE
case_id	uuid FK → cases(id) SET NULL nullable
subject / from_address / snippet	text nullable — denormaliserad cache
note	text nullable
(UNIQUE)	(gmail_thread_id, project_id)

messages

Kolumn	Typ / Constraint
id	uuid PK
project_id	uuid FK → projects(id) CASCADE
sender_id	uuid FK → auth.users SET NULL nullable
sender_name	text NOT NULL
sender_role	text NOT NULL — 'admin' 'customer'
body	text NOT NULL
created_at	timestampz default now()

settings

Kolumn	Typ / Constraint
key	text NOT NULL
value	text NOT NULL — krypteras ej, undvik känsliga värden i produktion
user_id	uuid FK → auth.users CASCADE
(PK)	(key, user_id) composite

Primär användning: `onedrive_refresh_token` per admin-användare.

4.3 Index

Index på	Motivering
projects(status)	Filtrera aktivitetsstatus i listvy
projects(deadline)	Hitta försenade projekt
cases(project_id)	Hämta alla ärenden per projekt
cases(status), cases(priority)	Filtrera ärendelistor
cases(deadline)	Daglig cron-query för försenade ärenden
documents(project_id)	Hämta alla filer per projekt
email_links(gmail_thread_id)	Snabb lookup vid e-postkoppling
messages(project_id)	Ladda chathistorik
portal_access(invite_token)	Inbjudningslänk-validering
portal_access(user_id)	Hämta kundens projekt vid inloggning

5. Autentisering och auktorisering

5.1 Auth-flöde – Admin

1. POST /auth/v1/token?grant_type=password (Supabase Auth)
2. Supabase returnerar access_token (JWT) + refresh_token
3. @supabase/ssr sparar token i HttpOnly-cookie
4. middleware.ts:
 - a. createServerClient() läser cookie
 - b. supabase.auth.getUser() → validerar JWT mot Supabase
 - c. Hämtar profiles.role → om inte 'admin' → redirect /portal
5. Server Components anropar createClient() från server.ts
→ Samma cookie → Samma JWT → RLS-context satt automatiskt

5.2 Auth-flöde – Kund (ny inbjudan)

1. Admin klickar 'Generera länk' på kontaktsidan
2. InviteButton.tsx anropar generateInviteAction()
→ Upsert i portal_access, returnerar invite_token
3. send-invite Edge Function anropas med JWT
→ Hämtar kontaktens e-post + projektnamn
→ Skickar välkomstmejl via Resend med länk:
/portal/invite/{token}
4. Kunden öppnar länken → /portal/invite/[token]/page.tsx
5. Supabase.auth.signUp() med kontaktens e-post
6. UPDATE portal_access SET user_id=..., accepted_at=now()
WHERE invite_token = token
7. UPDATE profiles SET role='customer'
8. Redirect → /portal

5.3 Row Level Security (RLS)

Alla tabeller har RLS aktiverat. Två helper-funktioner används i alla policies:

```
-- Kontrollerar om aktiv användare är admin
CREATE OR REPLACE FUNCTION is_admin()
RETURNS boolean AS $$
  SELECT EXISTS (
    SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin'
  );
$$ LANGUAGE sql SECURITY DEFINER STABLE;

-- Kontrollerar om aktiv användare har accepterad portalåtkomst till projektet
CREATE OR REPLACE FUNCTION has_project_access(p_project_id uuid)
RETURNS boolean AS $$
  SELECT EXISTS (
    SELECT 1 FROM portal_access
    WHERE user_id = auth.uid()
      AND project_id = p_project_id
      AND accepted_at IS NOT NULL
  );
$$ LANGUAGE sql SECURITY DEFINER STABLE;
```

Principen per tabell:

Tabell	RLS-princip
projects	Admin: ALL. Kund: SELECT WHERE has_project_access(id)
cases	Admin: ALL. Kund: SELECT WHERE has_project_access(project_id)
messages	Admin: ALL. Kund: SELECT + INSERT WHERE has_project_access(project_id)
documents	Admin: ALL. Kund: SELECT + INSERT WHERE has_project_access(project_id)
portal_access	Admin: ALL. Kund: SELECT WHERE user_id = auth.uid()
settings	Endast ägaren: ALL WHERE user_id = auth.uid()
email_links	Admin: ALL. Kunder ser inga e-postlänkar.
profiles	Användaren ser/redigerar sin egen. Admin ser alla.

5.4 Middleware-skydd

src/middleware.ts kör på varje request innan sidan renderas:

```
Skyddade admin-routes: /dashboard, /projects, /contacts, /cases, /email,
                        /documents
  → Ingen session → redirect /login

Skyddad kundportal:    /portal/project/*
  → Ingen session → redirect /portal/login

Redan inloggad:        /login eller /portal/login
  → Admin → redirect /dashboard
  → Kund  → redirect /portal

Undantag (ej skyddade): /portal/invite/*, /api/*, statiska filer
```

6. API-lager

6.1 Server Actions (src/lib/actions.ts)

Server Actions är Next.js 15-primitiven för formulärhantering. De körs exklusivt på servern, kan anropas direkt från RSC-formulär, och kör `revalidatePath()` + `redirect()` för optimistisk navigation utan extra API-anrop.

Alla actions inleds med `requireAdmin()` som verifierar session och admin-roll. Vid fail → `redirect /login`.

Action	Signatur / Beteende
<code>createProjectAction</code>	<code>(formData) → insert projects + optional milestones → redirect /projects/{id}</code>
<code>updateProjectAction</code>	<code>(id, formData) → update projects → revalidate + redirect</code>
<code>deleteProjectAction</code>	<code>(id) → delete projects (cascade) → redirect /projects</code>
<code>toggleMilestoneAction</code>	<code>(id, done, projectId) → update milestones.done → revalidate</code>
<code>createContactAction</code>	<code>(formData) → insert contacts → optional project_contacts → redirect</code>
<code>updateContactAction</code>	<code>(id, formData) → update contacts → revalidate</code>
<code>deleteContactAction</code>	<code>(id) → delete contacts → redirect /contacts</code>
<code>linkContactToProjectAction</code>	<code>(contactId, projectId) → upsert project_contacts</code>
<code>createCaseAction</code>	<code>(formData) → insert cases → revalidate projects + cases</code>
<code>updateCaseAction</code>	<code>(id, formData) → update cases → revalidate</code>
<code>updateCaseStatusAction</code>	<code>(id, status, projectId) → snabb statusuppdatering</code>
<code>deleteCaseAction</code>	<code>(id, projectId) → delete cases → revalidate</code>
<code>generateInviteAction</code>	<code>(contactId, projectId) → upsert portal_access → return token</code>
<code>linkEmailAction</code>	<code>(formData) → upsert email_links → revalidate project</code>

6.2 Route Handlers (src/app/api/)

Route	Metod / Funktion
<code>/api/auth/signout</code>	POST → <code>supabase.auth.signOut()</code> → redirect <code>/portal/login</code>
<code>/api/auth/onedrive/callback</code>	GET → <code>exchangeCodeForTokens()</code> → spara <code>refresh_token</code> i settings
<code>/api/gmail/send</code>	POST { to, subject, body, threadId } → MCP → Gmail send
<code>/api/documents/upload</code>	POST multipart → <code>Storage.upload()</code> + insert documents
<code>/api/onedrive/upload</code>	POST multipart → <code>getGraphToken()</code> → Graph PUT → insert documents

6.3 Queries (src/lib/supabase/queries.ts)

Alla databasoperationer är samlade i `queries.ts`. Varje funktion skapar sin egen server-klient (cookies) och returnerar typade resultat. Exempel på join-mönster:

```
// Hämtar projekt med milestones och contacts via junction-tabell
const { data } = await sb
  .from("projects")
  .select(`
    *,
    milestones(*),
    project_contacts(
      contacts(*)
    ),
    cases(*, contacts(*))
  `)
  .eq("id", id)
  .single();

// Transformerar junction-resultat
const contacts = data.project_contacts
  .map(pc => pc.contacts)
  .filter(Boolean);
```

7. Integrationer

7.1 Gmail via Anthropic MCP

Gmail-integrationen använder inte Gmail API direkt. Istället proxyas anrop via Anthropic-modellens Model Context Protocol (MCP) med den anslutna Gmail-connectorn i Claude.ai-sessionen.

```
// Generellt mönster i src/app/(admin)/email/page.tsx
const response = await fetch("https://api.anthropic.com/v1/messages", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    model: "claude-sonnet-4-20250514",
    max_tokens: 1000,
    mcp_servers: [{
      type: "url",
      url: "https://gmailmcp.googleapis.com/mcp/v1",
      name: "gmail"
    }],
    messages: [{ role: "user", content: "<instruktion till modellen>" }]
  })
});

const data = await response.json();

// Extrahera svar - kan vara text, mcp_tool_use eller mcp_tool_result
const toolResults = data.content
  .filter(b => b.type === "mcp_tool_result")
  .map(b => b.content?.[0]?.text || "");
const textBlocks = data.content
  .filter(b => b.type === "text")
  .map(b => b.text);

// JSON-extraktion via regex på sammanfogad text
const raw = [...toolResults, ...textBlocks].join("\n");
const match = raw.match(/\[([^\s\S]*)\]/);
const messages = match ? JSON.parse(match[0]) : [];
```

⚠ Begränsning

MCP-anrop görs client-side via fetch() mot Anthropic API – API-nyckeln injiceras av Claude.ai-miljön och behöver inte konfigureras.

Funktionaliteten kräver att användaren är inloggad på claude.ai med Gmail anslutet som connector. Gmail MCP-modulen är ett Client Component (CC) och kan inte köras server-side.

7.2 OneDrive via Microsoft Graph API

src/lib/onedrive.ts implementerar hela Graph-integrationen. OAuth-flödet är Authorization Code Grant med offline_access scope för refresh tokens.

```
// Filstruktur i OneDrive
/VA-Projekt/
  /{projectName}/
    /{timestamp}_{filename}
```

```
// Token-flöde
1. Admin klickar "Anslut Microsoft 365"
   → Redirect till /api/auth/onedrive/callback?code=...
2. exchangeCodeForTokens(code) → { access_token, refresh_token }
3. refresh_token sparas i settings-tabellen (key='onedrive_refresh_token')
4. Vid varje API-anrop: getGraphToken(refresh_token) → ny access_token
5. Filuppladdning (< 4 MB): PUT /me/drive/root/{path}/content
   Större filer: Resumable upload session (EJ IMPLEMENTERAT)
```

7.3 Resend – e-postutskick

Resend används uteslutande från Supabase Edge Functions (Deno-miljö). Alla mallar genererar fullständig HTML med inline-stilar (kompatibelt med e-postklienter).

```
// supabase/functions/_shared/email.ts
await fetch("https://api.resend.com/emails", {
  method: "POST",
  headers: {
    Authorization: `Bearer ${Deno.env.get("RESEND_API_KEY")}`,
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    from: `${fromName} <${fromAddress}>`,
    to: [recipient],
    subject: "...",
    html: "..."
  })
});
```


8. Edge Functions

8.1 Översikt

Function	Trigger / Auth
notify-overdue	pg_cron (0 8 * * 1-5) / x-cron-secret header
notify-case-update	DB Webhook cases:UPDATE / x-supabase-signature header
notify-new-message	DB Webhook messages:INSERT / x-supabase-signature header
send-invite	HTTP POST / Supabase JWT (Bearer token)

8.2 notify-overdue

```
Input:  POST request med x-cron-secret header
Query:  cases WHERE deadline < today AND status IN ('Öppen','Pågående')
        JOIN contacts (name, email), projects (name)
Output: Grupperare per e-postadress → ett samlings-mejl per kontakt
Logg:   INSERT function_logs (sent, errors, total_cases)
```

8.3 notify-case-update

```
Input:  Supabase Webhook-payload { record, old_record }
Guard:  Hoppas över om record.status === old_record.status
Query:  Hämtar project(name), contact(name, email), admin(full_name)
Output: E-post till kontaktens adress med ny status och ärendetitel
```

8.4 notify-new-message

```
Input:  Supabase Webhook-payload { record }
Logic:
  Om sender_role === 'customer':
    → Hämtar alla admin-users via supabase.auth.admin.listUsers()
    → Skickar till alla admins
  Om sender_role === 'admin':
    → Hämtar portal_access WHERE project_id = msg.project_id AND accepted_at
    IS NOT NULL
    → Skickar till alla godkända kunder på projektet
```

8.5 send-invite

```
Input:  POST { contactId, projectId } med Supabase JWT
Auth:   Verifierar JWT → hämtar profiles.role → kräver 'admin'
Steps:
  1. UPSERT portal_access ON CONFLICT (contact_id, project_id) DO NOTHING
  2. SELECT invite_token, contacts(name, email), projects(name)
  3. Skickar inviteTemplate() via Resend
Output: { sent: true, to: email, inviteUrl: string }
```


9. Frontend-arkitektur

9.1 Server Components vs Client Components

Systemet maximerar Server Components (RSC) för datahämtning och minimerar JavaScript på klienten. Client Components ('use client') används bara när det krävs:

Komponent	RSC eller CC – Motivering
dashboard/page.tsx	RSC – ren datahämtning, ingen interaktivitet
projects/page.tsx	RSC – listan renderas server-side
projects/[id]/page.tsx	RSC – alla joins görs server-side
projects/[id]/edit/MilestoneToggle	CC – useTransition för optimistisk UI
contacts/[id]/InviteButton	CC – async action + kopiera till urklipp
cases/[id]/StatusChanger	CC – useTransition utan form-submit
email/page.tsx	CC – fetch() mot Anthropic API, useState för inbox
portal/messages/MessagesClient	CC – Supabase Realtime WebSocket
portal/documents/DocumentUploader	CC – File drag-drop, fetch()
Sidebar	CC – usePathname() för aktiv route-highlight
FormFields.tsx	CC – useFormStatus() för pending-state på submit

9.2 State-hantering

Ingen global state manager (Redux, Zustand etc.). State hanteras på tre nivåer:

- Server-state: RSC + revalidatePath() i Server Actions – Next.js-cachen är source of truth
- Komponent-lokal state: useState, useTransition för optimistiska uppdateringar
- Realtime-state: Supabase-kanal i MessagesClient – appends till lokal useState-array

9.3 Realtid – Supabase Realtime

```
// MessagesClient.tsx - abonnerar på nya meddelanden i ett projekt
useEffect(() => {
  const channel = supabase
    .channel(`messages:${projectId}`)
    .on("postgres_changes", {
      event: "INSERT",
      schema: "public",
      table: "messages",
      filter: `project_id=eq.${projectId}`,
    }, payload => {
      setMessages(prev => {
        if (prev.some(m => m.id === payload.new.id)) return prev; // dedup
        return [...prev, payload.new as Message];
      });
    })
    .subscribe();

  return () => { supabase.removeChannel(channel); };
}, [projectId]);
```

```
// Realtime är aktiverat för: messages, cases, milestones
// Konfigurerat i 02_rls storage.sql:
-- ALTER PUBLICATION supabase_realtime ADD TABLE messages;
```

9.4 Formulärhantering

```
// Mönster: RSC-formulär → Server Action
// Formulär i RSC (ingen 'use client' behövs)
<form action={createProjectAction}>
  <input name="name" required />
  <SubmitButton label="Skapa" /> { /* CC - hanterar pending-state */ }
</form>

// SubmitButton.tsx - CC med useFormStatus
const { pending } = useFormStatus();
return (
  <button type="submit" disabled={pending}>
    {pending ? "Sparar..." : label}
  </button>
);
```

9.5 Design-system

Inga externa komponentbibliotek. Allt är custom-byggt med Tailwind utility classes och globala CSS-klasser definierade i globals.css:

```
.btn-primary    → bg-[#1E4D8C] text-white rounded-lg ...
.btn-secondary → border border-[#DDDBD6] bg-white ...
.card           → bg-white border border-[#DDDBD6] rounded-xl p-5
.input          → border bg-[#F4F3EF] rounded-lg focus:border-[#1E4D8C]
.label          → text-[11px] font-bold text-[#7A7870] uppercase tracking-wider
.badge          → inline-flex items-center gap-1 px-2.5 py-0.5 rounded-full
.page-title     → text-2xl font-bold font-display text-[#1A1916]
```

Typografi via Google Fonts (next/font/google):

- DM Sans – UI-text, labels, knappar (var: --font-dm-sans)
- Playfair Display – rubriker, hero-text (var: --font-playfair)

10. Säkerhetsmodell

10.1 JWT och session

Aspekt	Implementation
Token-lagring	HttpOnly-cookies via @supabase/ssr – ej localStorage
Token-rotation	Supabase roterar access_token automatiskt vid expiry
CSRF	Next.js Server Actions inkluderar CSRF-skydd inbyggt
XSS	HttpOnly cookies + React DOM-escaping

10.2 Hemlighetsshantering

Typ	Lagring
SUPABASE_URL, ANON_KEY	Vercel env vars (NEXT_PUBLIC_ – exponeras till klient)
SERVICE_ROLE_KEY	Vercel env vars (server-only, aldrig till klient)
RESEND_API_KEY	Supabase secrets (Deno.env i Edge Functions)
MICROSOFT_CLIENT_SECRET	Vercel env vars (server-only)
OneDrive refresh_token	settings-tabellen i Supabase (DB-lagring, RLS-skyddad)
CRON_SECRET	Supabase secrets – autentiserar pg_cron → Edge Function
SUPABASE_WEBHOOK_SECRET	Supabase secrets – autentiserar DB Webhook → Edge Function

⚠ Säkerhetsvarning

SERVICE_ROLE_KEY kringgår RLS helt. Används endast i Edge Functions (Deno.env) och aldrig i client-side kod.

OneDrive refresh_token lagras i klartext i settings-tabellen. I en produktionsmiljö bör detta krypteras med pgcrypto.

10.3 Storage-säkerhet

Supabase Storage-bucketen project-documents är privat (public: false). Åtkomst via:

- Signerade URL:er med 3600s TTL – genereras server-side via createSignedUrl()
- RLS på storage.objects: admin kan allt, kund kan läsa/skriva i sin projektmapp
- Sökvägskonvention: {project_id}/{timestamp}_{filename} – project_id extraheras ur sökvägen för RLS-kontroll

11. Utökningsmönster

11.1 Lägga till en ny databastabell

```
-- 1. Lägg till i schema-SQL
CREATE TABLE invoices (
  id          uuid PRIMARY KEY DEFAULT uuid_generate_v4(),
  project_id  uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  amount      numeric NOT NULL,
  due_date    date,
  paid        boolean DEFAULT false,
  created_at  timestamptz DEFAULT now()
);

-- 2. Aktivera RLS
ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Admin full access" ON invoices FOR ALL USING (is_admin());

-- 3. Lägg till typ i src/lib/supabase/types.ts
export interface Invoice {
  id: string; project_id: string; amount: number;
  due_date: string | null; paid: boolean; created_at: string;
}

-- 4. Lägg till queries i src/lib/supabase/queries.ts
export async function getInvoices(projectId: string): Promise<Invoice[]> {
  const sb = await createClient();
  const { data, error } = await sb.from("invoices")
    .select("*").eq("project_id", projectId).order("due_date");
  if (error) throw error;
  return data ?? [];
}

-- 5. Lägg till Server Action i src/lib/actions.ts
export async function createInvoiceAction(formData: FormData) {
  const { sb } = await requireAdmin();
  const projectId = str(formData.get("projectId"));
  await sb.from("invoices").insert({ project_id: projectId, amount: ... });
  revalidatePath(`/projects/${projectId}`);
  redirect(`/projects/${projectId}`);
}

-- 6. Skapa sida: src/app/(admin)/projects/[id]/invoices/page.tsx
```

11.2 Lägga till en ny admin-route

```
-- Filsystemet IS routing i Next.js App Router

src/app/(admin)/reports/
  page.tsx      ← /reports (RSC, datahämtning)
  new/page.tsx  ← /reports/new (formulär)
  [id]/page.tsx ← /reports/{id}

-- Lägg till i Sidebar.tsx:
const NAV = [
  ...
  { href: "/reports", icon: "📊", label: "Rapporter" },
];
```

```
-- Inga extra routing-filer behövs.  
-- (admin)-grupplayouten och middleware skyddar automatiskt.
```

11.3 Lägga till en ny Edge Function

```
-- 1. Skapa mapp och fil  
supabase/functions/my-function/index.ts  
  
-- 2. Boilerplate  
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";  
import { sendEmail } from "../_shared/email.ts";  
  
Deno.serve(async (req) => {  
  // Auth check  
  const secret = req.headers.get("x-cron-secret");  
  if (secret !== Deno.env.get("CRON_SECRET")) {  
    return new Response("Unauthorized", { status: 401 });  
  }  
  
  const sb = createClient(  
    Deno.env.get("SUPABASE_URL")!,  
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!  
  );  
  
  // ... logik ...  
  
  return new Response(JSON.stringify({ ok: true }), {  
    headers: { "Content-Type": "application/json" }  
  });  
});  
  
-- 3. Deploya  
supabase functions deploy my-function --no-verify-jwt  
  
-- 4. Sätt upp trigger (webhook eller pg_cron) i Supabase Dashboard
```

11.4 Lägga till e-postmall

```
-- I supabase/functions/_shared/email.ts:  
  
export function myTemplate(data: {  
  recipientName: string;  
  content: string;  
  actionUrl: string;  
}): string {  
  return `  
    <div style="...baseStyle...">  
      <div style="...cardStyle...">  
        ${header("Min rubrik")}  
        <div style="padding: 28px 32px;">  
          <p>Hej ${data.recipientName},</p>  
          <p>${data.content}</p>  
          ${button("Gå till systemet", data.actionUrl)}  
        </div>  
        ${footer()}  
      </div>  
    </div>  
  `;  
}
```

```
}

-- Anropas sedan i din Edge Function:
await sendEmail({
  to: contact.email,
  subject: "Min rubrik",
  html: myTemplate({ ... })
});
```

11.5 Utöka kundportalen

```
-- Ny flik i kundportalen:

-- 1. Lägg till i PortalNav.tsx:
const tabs = [
  ...
  { href: `${base}/timeline`, label: "Tidslinje" },
];

-- 2. Skapa sida:
src/app/portal/project/[id]/timeline/page.tsx

-- 3. Verifiera att RLS tillåter kunden att läsa den data sidan behöver
(has_project_access() är already satt på alla relevanta tabeller)

-- 4. Sidan ärver auth-check från:
src/app/portal/project/[id]/layout.tsx
som verifierar portal_access med accepted_at IS NOT NULL
```


12. Miljövariabler – komplett referens

12.1 Next.js / Vercel (.env.local)

Variabel	Typ / Kontext / Källa
NEXT_PUBLIC_SUPABASE_URL	Public. Klient + server. Supabase → Settings → API → Project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public. Klient + server. Supabase → Settings → API → anon public
NEXT_PUBLIC_APP_URL	Public. Klient + server. Din Vercel-URL (https://...)
MICROSOFT_CLIENT_ID	Server-only. Azure → App registrations → Application ID
MICROSOFT_CLIENT_SECRET	Server-only. Azure → App registrations → Certificates & secrets
MICROSOFT_TENANT_ID	Server-only. Azure → App registrations → Directory ID
MICROSOFT_REDIRECT_URI	Server-only. {APP_URL}/api/auth/onedrive/callback

12.2 Supabase Edge Functions (supabase secrets set)

Variabel	Syfte
SUPABASE_URL	Automatiskt injicerad av Supabase runtime
SUPABASE_SERVICE_ROLE_KEY	Automatiskt injicerad av Supabase runtime
RESEND_API_KEY	Autentisering mot Resend API
EMAIL_FROM_NAME	Avsändarnamn i utgående mejl
EMAIL_FROM_ADDRESS	Avsändaradress (måste vara verifierad i Resend)
NEXT_PUBLIC_APP_URL	Bas-URL för länkar i mejlmallar
CRON_SECRET	Validerar att anrop till notify-overdue kommer från pg_cron
SUPABASE_WEBHOOK_SECRET	Validerar att Webhook-anrop kommer från Supabase

13. Kända begränsningar och framtida arbete

13.1 Säkerhet och tech debt

Issue	Beskrivning / Åtgärd
OneDrive refresh_token i klartext	settings-tabellen lagrar token okrypterat. Åtgärd: pgcrypto.encrypt() med en applikationsnyckel.
En enda admin	Systemet är designat för en admin. Multi-admin kräver ny RLS-logik och team-kontext i URL.
Gmail är client-side MCP	Kräver Claude.ai-session. Bör ersättas med server-side OAuth om Claude.ai-beroende är ett problem.
Inga rate limits	API-routes saknar rate limiting. Bör läggas till via Vercel Edge Middleware eller Upstash.
Inga migrations	SQL-filer körs manuellt. Bör migreras till supabase migrations-systemet (supabase db push).

13.2 Saknade funktioner (backlog)

- Sökfunktion – fulltextsökning i projekt, ärenden och kontakter (PostgreSQL full-text search är tillgängligt)
- PDF-rapportgenerering – t.ex. exportera ett projekts ärenden till PDF
- Filtrering och sortering – mer avancerade filter i ärendelistan (datum-range, projektstatus-kombination)
- Aviseringar in-app – toast-notifikationer för realtidshändelser utan e-post
- Multi-admin – stöd för konsulter eller assistenter med begränsad admin-access
- Resumable file upload – OneDrive-uppladdning för filer > 4 MB (Graph API stöder detta)
- Offline-support – Service Worker för att cacha projektlistan vid dålig uppkoppling
- Exportfunktion – CSV-export av kontakter och ärenden
- Audit log – loggning av alla create/update/delete-händelser per projekt
- Tidrapportering – koppla timmar mot projekt och ärenden

13.3 Skalningsgränser

Komponent	Nuvarande gräns / Åtgärd vid behov
Supabase Free tier	500 MB databas, 1 GB storage, 2 GB bandbredd. Uppgradera till Pro (\$25/mån) vid behov.
Vercel Hobby	100 GB bandbredd, 6000 build-minuter/mån. Uppgradera vid hög trafik.
Resend Free	3 000 mejl/mån. Uppgradera till \$20/mån för 50 000 mejl.
Supabase Realtime	Max 200 samtidiga anslutningar på Free. Räcker gott för ett 30-projekts-system.
MCP/Gmail	Begränsad av Anthropic API rate limits. Ej lämplig för batch-operationer.